

Desenvolvimento de um Sistema Web Completo com Framework MVC e Controle de Versão Git

Vocês devem construir, **do início ao fim**, um sistema web real usando o *Framework MVC Didático em PHP* apresentado em aula. O sistema é uma clínica veterinária: cadastros, relacionamentos entre tabelas, controle de acesso com tela de login, pesquisa, relatórios em PDF, regras de negócio próprias e testes automatizados. Todo o desenvolvimento deve ser feito **dentro de um repositório Git**, com histórico limpo, branches por funcionalidade e commits que contem a história do projeto. **O que será avaliado não é só o sistema pronto: é também o caminho percorrido até ele.**

1. Objetivos de aprendizagem

Ao concluir esta atividade, você deve ser capaz de:

- Traduzir um pedido escrito em linguagem de negócio para um **modelo de dados** (tabelas, campos, tipos e relacionamentos);
- Construir uma aplicação seguindo o padrão **MVC**, sabendo o que pertence ao model, ao controller e à view;
- Usar geradores de código com critério — **lendo e adaptando** o que foi gerado, em vez de aceitá-lo cegamente;
- Implementar **autenticação e autorização**, entendendo a diferença entre “quem é você” e “o que você pode fazer”;
- Aplicar práticas básicas de **segurança web**: token anti-CSRF, escape de saída, consultas parametrizadas;
- Escrever **testes automatizados** que provem que uma regra de negócio funciona;
- Usar o **Git** como ferramenta diária de trabalho: branches, merges, conflitos, tags e um histórico legível.

2. O contexto: o pedido do cliente

A **Clínica Veterinária Pata Amiga** atende cães, gatos e pequenos animais em um bairro da cidade. Hoje a clínica controla tudo em cadernos e em uma planilha que só a recepcionista sabe usar. Os problemas relatados pela dona da clínica foram estes:

- “Quando o tutor liga perguntando quando o bicho tomou a última vacina, a gente demora dez minutos folheando o caderno.”
- “Já aconteceu de marcar dois animais para o mesmo veterinário no mesmo horário.”
- “No fim do mês eu não sei dizer quantos atendimentos foram feitos nem quanto entrou.”
- “Os tutores vivem pedindo a carteirinha de vacinação impressa.”
- “Hoje qualquer um que senta no computador da recepção mexe em tudo, e depois ninguém sabe quem lançou o quê.”

A clínica contratou **você**. As seções seguintes descrevem *o que* o sistema precisa fazer. **Como** fazer — quais tabelas criar, com quais campos e tipos, quais comandos usar, como organizar as telas — é decisão sua, e é exatamente isso que está sendo avaliado.

Leia com atenção

Os requisitos abaixo estão escritos em **linguagem de negócio, não em linguagem de banco de dados**. Em nenhum momento eles dizem “crie a tabela X com o campo Y do tipo Z”. Essa tradução é o seu trabalho. Duas soluções diferentes podem estar as duas corretas — desde que atendam ao que foi pedido e você saiba justificar suas escolhas.

3. Requisitos funcionais

Módulo A — Cadastros de apoio

Cód.	O sistema deve permitir que a equipe...
RF01	cadastre as espécies atendidas (cão, gato, ave, roedor...), com nome e observações de manejo. Não podem existir duas espécies com o mesmo nome.
RF02	cadastre os tutores (donos dos animais), guardando nome, CPF, telefone, e-mail, endereço e a data em que se tornou cliente. Não podem existir dois tutores com o mesmo e-mail.
RF03	cadastre os veterinários , com nome, CRMV, especialidade, telefone e uma marcação de <i>ativo</i> — profissionais inativos não devem aparecer para escolha em novos agendamentos.
RF04	cadastre os procedimentos oferecidos (consulta de rotina, vacinação, castração, banho...), com descrição, valor de tabela e duração estimada em minutos.

Módulo B — Animais

Cód.	O sistema deve...
RF05	registrar os animais . Todo animal pertence a exatamente um tutor e a uma espécie . Guardar ainda: nome, raça, data de nascimento, sexo, peso em quilos, se é castrado e observações gerais.
RF06	impedir o cadastro de um animal sem tutor ou sem espécie. A escolha do tutor e da espécie deve ser feita em uma lista, nunca digitando um número.
RF07	exibir, na tela de detalhes do animal, os dados do tutor, a idade calculada a partir da data de nascimento e o histórico de atendimentos e vacinas daquele animal.
RF08	oferecer, na listagem de animais, uma pesquisa por nome, por espécie e por tutor. Os filtros preenchidos devem se combinar; a listagem completa aparece enquanto ninguém pesquisa.

Módulo C — Atendimentos

Cód.	O sistema deve...
RF09	registrar atendimentos , relacionando um animal, um veterinário e um procedimento, com data e hora, valor efetivamente cobrado, observações clínicas e uma situação (agendado, realizado ou cancelado).
RF10	recusar o agendamento de um atendimento para um veterinário que já tenha outro atendimento no mesmo

	dia e horário — mostrando uma mensagem clara, sem perder o que já foi digitado no formulário. <i>(regra de negócio, não sai pronta de nenhum gerador)</i>
RF11	recusar o agendamento com data no passado. Um atendimento já <i>realizado</i> , no entanto, pode ser registrado com data anterior.
RF12	oferecer, na listagem de atendimentos, pesquisa por data, por veterinário e por situação.
RF13	exibir, na listagem filtrada, a quantidade de atendimentos e a soma dos valores correspondentes ao filtro aplicado.

Módulo D — Vacinas

Cód.	O sistema deve...
RF14	registrar as vacinas aplicadas em cada animal: nome da vacina, lote, data de aplicação, data prevista de retorno e o veterinário responsável.
RF15	destacar na página inicial da equipe os retornos de vacina <i>vencidos</i> e os que vencem <i>nos próximos 30 dias</i> , cada grupo com sua contagem e com link para o animal.

Módulo E — Acesso e permissões

Cód.	O sistema deve...
RF16	ter uma tela de login para a equipe da clínica . Só quem tem conta usa o sistema; os tutores são <i>cadastrados</i> pela equipe, mas não acessam o sistema.
RF17	garantir que nenhuma rota de cadastro, edição, exclusão ou relatório fique acessível sem login. Um visitante não autenticado que tente abrir uma dessas telas deve ser levado ao login correspondente.
RF18	gravar em cada atendimento e em cada vacina qual usuário da equipe fez o lançamento , e mostrar essa autoria na tela de detalhes. O dado deve vir da sessão de quem está logado — nunca de um campo do formulário, que qualquer um poderia forjar. <i>(regra de negócio; nenhum gerador faz isso por você)</i>
RF19	oferecer a saída da sessão pelo menu. Depois de sair, nenhuma tela protegida pode ser reaberta pelo endereço direto ou pelo botão voltar do navegador.
RF20	exibir um menu coerente com a situação : quem está logado vê os cadastros e a saída; o visitante vê apenas a apresentação e o link de entrada.

Módulo F — Relatórios e apresentação

Cód.	O sistema deve...
RF21	gerar um relatório em PDF da agenda , filtrável por data e por veterinário.
RF22	gerar um PDF da carteira de vacinação de um animal, contendo os dados do animal, do tutor e todas as vacinas aplicadas.

RF23	apresentar datas no formato brasileiro (dd/mm/aaaa) e valores em reais (1.234,50) em todas as telas e relatórios. Campos de sim/não nunca devem aparecer como 1 ou 0.
RF24	confirmar cada operação bem-sucedida com uma mensagem e, em caso de erro de preenchimento, devolver o formulário com as mensagens por campo e os dados já digitados.
RF25	ter uma página inicial pública apresentando a clínica, com o acesso da equipe claramente indicado.

4. Requisitos técnicos

Cód.	Exigência
RT01	Usar o framework fornecido em aula , com PHP 8.1 ou superior e MySQL/MariaDB. Não é permitido instalar bibliotecas externas, usar Composer ou trocar por outro framework.
RT02	Os geradores do <code>console.php</code> são o ponto de partida , não a entrega. Todo arquivo gerado deve ser lido e ajustado ao domínio (rótulos, ordem das colunas, validações, textos). Entregar telas com rótulos genéricos do gerador é entregar trabalho pela metade.
RT03	Regras de negócio ficam no model (em <code>validar()</code> ou em métodos próprios). Controllers devem apenas receber, delegar e responder. Nenhuma consulta SQL escrita dentro de uma view.
RT04	Todo formulário deve conter o campo de token anti-CSRF e o controller deve validá-lo. Exclusões só por <code>POST</code> .
RT05	Todo dado vindo do banco impresso em tela deve ser escapado . Nenhuma variável do usuário concatenada dentro de uma string SQL — apenas parâmetros vinculados.
RT06	Senhas nunca em texto puro no banco. Verifique isso olhando a tabela diretamente antes de entregar.
RT07	O arquivo de esquema do banco deve estar versionado e ser suficiente para recriar o sistema do zero em outra máquina, sem nenhum passo manual no MySQL.
RT08	Todos os testes devem passar na entrega. Além dos gerados, escreva no mínimo 4 testes próprios , cobrindo obrigatoriamente as regras RF10, RF11 e RF18.
RT09	Nenhum aviso ou erro de PHP visível nas telas; <code>php -l</code> sem erros em todos os arquivos do projeto.
RT10	Nomes de tabelas, colunas, classes e variáveis em português, coerentes entre si , seguindo o padrão já usado pelo framework.

5. Uso obrigatório do Git

Um projeto entregue com um único commit no fim recebe zero nesta seção, mesmo que o sistema funcione perfeitamente. O histórico é parte da entrega: ele mostra *como* você trabalhou, e é assim que se trabalha em equipe no mercado.

Cód.	Exigência
RGo1	O projeto vive em um repositório Git próprio , criado antes de gerar a primeira linha de código, com um commit inicial contendo apenas o framework limpo.
RGo2	Repositório remoto (GitHub ou GitLab) com o professor adicionado como colaborador desde a primeira semana. <i>Envie o link assim que criar — não na véspera da entrega.</i>
RGo3	Arquivo <code>.gitignore</code> revisado por você: arquivos gerados, PDFs de saída, configurações locais de senha e lixo do sistema operacional não devem entrar no repositório. Justifique suas escolhas no relatório.
RGo4	Mínimo de 25 commits significativos, distribuídos em pelo menos 5 dias diferentes . As datas ficam registradas no histórico: um projeto inteiro commitado na véspera é evidente e será penalizado.
RGo5	Mensagens de commit no padrão Conventional Commits , em português, no imperativo e descrevendo o <i>efeito</i> da mudança: feat: agendar atendimento com validação de horário / fix: gravar a autoria do atendimento pela sessão Prefixos aceitos: feat, fix, docs, refactor, test, style, chore. Mensagens como “ajustes”, “att”, “commit 3” ou “funcionando” não contam para o mínimo do RGo4.
RGo6	A branch <code>main</code> deve estar sempre funcionando . Nenhum desenvolvimento é feito diretamente nela.
RGo7	Mínimo de 6 branches de funcionalidade — uma por módulo — nomeadas com sentido (feat/cadastros-base, feat/atendimentos, fix/idade-do-animal) e integradas à <code>main</code> por merge .
RGo8	Resolver pelo menos um conflito de merge real e descrever no relatório o que conflitou e por que você escolheu aquela resolução. <i>Trabalhando em dupla isso acontece sozinho; sozinho, provoque-o deliberadamente.</i>
RGo9	Usar pelo menos uma vez , de forma justificada e comentada no relatório, um comando de correção de histórico: <code>git revert</code> , <code>git restore</code> ou <code>git reset</code> .
RG10	Marcar as entregas parciais com tags : <code>v0.1</code> ao fim da Etapa 2, <code>v0.5</code> ao fim da Etapa 4 e <code>v1.0</code> na entrega final.
RG11	O <code>README.md</code> do repositório deve ser reescrito para o seu sistema (o do framework não serve): o que o sistema faz, como instalar do zero, os comandos usados na geração, o modelo de dados, os usuários de teste e os autores.
RG12	Em dupla : os dois integrantes devem ter commits próprios e proporcionais. Um <code>git shortlog -sn</code> mostrando 30 commits de um e 2 do outro resulta em notas diferentes para cada um.

6. Etapas sugeridas e checkpoints

Você não precisa seguir esta ordem, mas ela reduz retrabalho: cada etapa se apoia na anterior. Marque a data acordada com o professor para cada checkpoint.

Etapa	O que fica pronto	Branch sugerida	Tag	Data
0	Ambiente instalado, banco criado,	<code>main</code>	—	

	repositório iniciado e enviado ao remoto, <code>.gitignore</code> revisado			
1	Modelagem no papel: entidades, campos, tipos e relacionamentos desenhados e discutidos com o professor — <i>antes de gerar qualquer código</i>	<code>docs/modelagem</code>	—	
2	Módulo A completo: cadastros de apoio funcionando, telas ajustadas, menu organizado	<code>feat/cadastros-base</code>	<code>v0.1</code>	
3	Módulo B: animais com seus dois relacionamentos, pesquisa na listagem e tela de detalhes	<code>feat/animais</code>	—	
4	Módulo E: a tela de login da equipe, rotas protegidas, saída da sessão e menu por situação	<code>feat/acesso</code>	<code>v0.5</code>	
5	Módulo C: atendimentos, com as regras de conflito de horário e data no passado	<code>feat/atendimentos</code>	—	
6	Módulo D: vacinas e os alertas de retorno na página inicial	<code>feat/vacinas</code>	—	
7	Módulo F: relatórios em PDF, formatação brasileira, testes próprios, README e relatório final	<code>feat/relatorios</code>	<code>v1.0</code>	

Conselho de quem já corrigiu muitos desses

Faça a Etapa 4 antes das etapas 5 e 6. Colocar autenticação em um sistema que já tem cinco módulos prontos dá muito mais trabalho do que gerar os módulos seguintes já protegidos. Erros de ordem custam horas — e aparecem no histórico do Git.

7. Entregáveis

Item	Descrição
E1	Link do repositório remoto , com o professor como colaborador e a tag <code>v1.0</code> criada. É a entrega principal — nada de arquivos <code>.zip</code> .
E2	Relatório em PDF , de 3 a 5 páginas, versionado junto com o projeto, contendo: (a) o modelo de dados final, em diagrama ou tabela, com os relacionamentos;

	(b) a lista de comandos usados para gerar o sistema, na ordem em que foram executados; (c) as decisões que você tomou e por quê (nomes, tipos, o que ficou em cada camada); (d) como cada regra de negócio (RF10, RF11, RF18) foi implementada, com o trecho de código; (e) o que deu errado no caminho e como você resolveu.
E3	Captura da execução dos testes , mostrando todos passando, incluída no relatório.
E4	Vídeo de até 6 minutos (link) demonstrando o sistema em uso: login da equipe, um cadastro completo, a regra de conflito de horário sendo recusada, a autoria do lançamento aparecendo na tela de detalhes e os dois PDFs sendo gerados.
E5	Defesa oral de 10 minutos, na data marcada.

8. Desafios Bônus

- Painel na página inicial da equipe com indicadores do mês: atendimentos realizados, faturamento, animais novos e vacinas pendentes.
- Filtro de atendimentos por *período* (data inicial e data final), que exige ir além do que o gerador de pesquisa produz.
- Exportação da listagem filtrada em CSV, aberta corretamente no Excel com acentuação.
- Foto do animal, com upload validado por tipo e tamanho, exibida na tela de detalhes e na carteira de vacinação.

9. Regras de conduta

- **Código copiado de colegas** zera a atividade para todos os envolvidos. O histórico do Git torna isso fácil de identificar: dois repositórios com os mesmos commits na mesma ordem não são coincidência.
- **Uso de IA generativa** é permitido como ferramenta de apoio, desde que declarado no relatório: para que você a usou e o que verificou depois. O que **não** é permitido é entregar código que você não sabe explicar — e a defesa oral vai verificar exatamente isso.
- **Dúvidas sobre o enunciado** devem ser levadas ao professor durante as aulas, e não na véspera. Uma interpretação razoável, declarada no relatório, é sempre aceita.
- **Prazos parciais** existem para o seu benefício. Chegar à Etapa 4 sem ter feito a 1 significa refazer trabalho.

10. Anexo A — Referência rápida do framework

Os comandos abaixo são a ferramenta, não a solução. **Quais** executar, com quais argumentos e em que ordem, é o que você precisa decidir.

```
# preparar o ambiente e o banco
php instalar.php

# ver a ajuda completa do console
php console.php

# gerar um CRUD (model, controller, views, testes, tabela)
php console.php scaffold:crud <tabela> <campo:tipo> ... [--auth] [--modelo=Nome] [--sem-menu]
```

```
# tipos aceitos
string text integer decimal boolean date datetime time
<campo>_id:belongs_to=<tabela_pai> # relação 1:N (o pai precisa existir antes)

# pesquisa na listagem de um recurso já gerado
php console.php scaffold:pesquisa <tabela> <campo> [campo2 ...] [--remove]

# tela de login – o prefixo das rotas sai do próprio modelo
php console.php auth:install [Modelo|tabela] [Prefixo]

# relatório em PDF pelo terminal
php console.php relatorio:pdf <tabela|Modelo> [caminho/saida.pdf]

# testes
php testes/executar.php
php testes/executar.php NomeDoTeste::metodo

# servidor de desenvolvimento
php -S localhost:8000 roteador.php
```

Três armadilhas conhecidas

- A tela de login precisa existir **antes** de usar `--auth` em um scaffold.
- Em uma relação 1:N, a **tabela pai precisa ser criada primeiro**.
- Sem `--auth`, **todas** as rotas do recurso nascem públicas — inclusive excluir e o relatório.

Auxiliares que você vai precisar

Estes já existem no framework — procurá-los antes de reinventá-los economiza horas. A lista completa está no tutorial e na referência de comandos, dentro da pasta de documentação.

Nas views	Para quê
<code>e(\$valor)</code>	escapar HTML — use em tudo que vier do banco
<code>campo_csrf()</code>	o campo escondido com o token, obrigatório em todo formulário
<code>data_br()</code> / <code>moeda_br()</code> / <code>sim_ao()</code>	formatação brasileira exigida pelo RF23
<code>antigo()</code> / <code>erro_de()</code> / <code>tem_erro()</code>	devolver o formulário com os erros e o que já foi digitado
<code>autenticado()</code> / <code>usuario_id()</code>	quem está logado — base do menu (RF20) e da autoria exigida pelo RF18
<code>url()</code> / <code>asset()</code> / <code>parcial()</code>	endereços de rotas e arquivos, e inclusão de pedaços de view
<code>mensagens()</code> / <code>dd()</code>	mensagens de sucesso e erro; despejo de um valor durante a depuração

No controller	Para quê
<code>exigirAutenticacao()</code>	manda para o login quem não entrou — vale só no método onde é chamada
<code>exigirFormularioValido()</code>	exige POST e token válido de uma vez
<code>voltarComErros(\$erros, \$rota)</code>	devolve ao formulário com as mensagens por campo

11. Anexo B — Fluxo Git de referência

```
# uma única vez, no começo do projeto
git init
git add .
git commit -m "chore: adicionar framework base do projeto"
git branch -M main
git remote add origin <url-do-seu-repositorio>
git push -u origin main

# para CADA módulo: sair da main, trabalhar, voltar
git switch -c feat/cadastrros-base
... gerar, ajustar, testar ...
git add modelos/Especie.php controllers/EspeciesController.php
git commit -m "feat: cadastrar espécies atendidas pela clínica"
... mais alguns commits pequenos ...
git push -u origin feat/cadastrros-base

# integrar o módulo pronto e testado
git switch main
git merge --no-ff feat/cadastrros-base
git push

# marcar a entrega parcial
git tag -a v0.1 -m "Cadastrros de apoio concluídos"
git push origin v0.1

# inspecionar o histórico (a captura do relatório sai daqui)
git log --oneline --graph --all
git shortlog -sn
```

Um commit bom cabe em uma frase

Se a mensagem do seu commit precisa da palavra “e” duas vezes, provavelmente ele deveria ter sido dois commits. Faça commits **pequenos e temáticos**: um por comportamento que passou a funcionar. Isso não é burocracia — é o que permite descobrir depois *quando* algo quebrou.